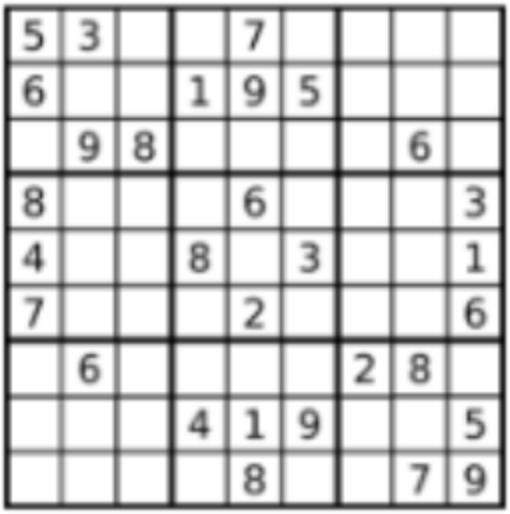
Prompt Used

• Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number.
• Each solutionN.txt file should include the following:
o If no solutions are found, a single line: “No solution found.”
o If solutions are found, it should include all solutions followed by a single line: “Solutions found: n”, where n is the number of solutions found.
o The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.
• You must modify the Assignment 4 you receive, not your Assignment 4.
• Do not replace the entire file with your code. Your GTA will use the Github Blame button to check your modifications.
• Add your name to the Profile Comments as “Revised by:” along with a revision date.
• You must get all required functionality from Assignment 4 working, even if the student you inherited it from did not

Assignment 4 requirements below:
C++ code using objects, not functions, to implement the following requirements
• Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
• A typical puzzle is shown here (image credit: Wikimedia Commons):
o 

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.

o Each digit from 1 to 9 must appear exactly once in each block.
• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it’s possible to gradually fill in the entire grid.
• That is not the approach we’re going to use. Instead, we’re going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by “slower to run,” we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
• Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five

puzzle files.
• The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
• To help you debug your program, solution1.txt is a solution to puzzle1.txt.
(all puzzle.txt's attached to both ais)

Code to modify below:

```
/*
 * Program Name: EECS 348 Assignment 4-Sudoku Solver
 * Description: This program looks at five Sudoku puzzle files and uses
recursive
 * depth-first search with backtracking to find every valid solution that
there is.
 * The program makes sure it checks rows, columns, and 3x3 blocks before
placing
 * a number in an empty cell.
 * Inputs:puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt.
 * Each file has to have a 9x9 Sudoku puzzle.
 * Outputs: The filename, original puzzle, and every solution found.
 * Author: Meklit Zewdie
 * Collaborators: ChatGPT and Gemini
 * Creation Date: October 15, 2026
 */

#include <iostream>
#include <fstream>
#include <string>
#include <cctype>
using namespace std;
class SudokuSolver
{
private:
    //A Sudoku board should have nine rows and nine columns
    int board[9][9];

    //This counts the number of solutions found for the puzzle.
    int solutionCount;
    //This checks if a number can be placed in a cell.
    bool isValid(int row, int col, int number)
```

```
{
    //Checking the cell's row.
    for (int i = 0; i < 9; i++)
    {
        if (board[row][i] == number)
        {
            return false;
        }
    }

    // This is checking the cell's column
    for (int i = 0; i < 9; i++)
    {
        if (board[i][col] == number)
        {
            return false;
        }
    }

    //Find the top-left corner of the cell's 3x3 block.
    int blockRow = (row / 3) * 3;
    int blockCol = (col / 3) * 3;

    // Check all nine cells in that 3x3 block.
    for (int r = blockRow; r < blockRow + 3; r++)
    {
        for (int c = blockCol; c < blockCol + 3; c++)
        {
            if (board[r][c] == number)
            {
                return false;
            }
        }
    }

    // The number does not go against with any Sudoku rule.
    return true;
}

//This checks whether the original clues contradict Sudoku rules.
bool hasValidStartingClues()
{
    for (int row = 0; row < 9; row++)
    {
        for (int col = 0; col < 9; col++)
        {
```

```
                int number = board[row][col];

                // Empty cells do not need to be checked.
                if (number == 0)
                {
                    continue;
                }

                //This part temporarily remove the clue so that isValid does
not
                // detect the number in the cell being checked.
                board[row][col] = 0;
                bool valid = isValid(row, col, number);
                board[row][col] = number;

                if (!valid)
                {
                    return false;
                }
            }
        }

        return true;
    }

    //Prints the current Sudoku board.
    void printBoard()
    {
        for (int row = 0; row < 9; row++)
        {
            for (int col = 0; col < 9; col++)
            {
                cout << board[row][col] << " ";
            }

            cout << endl;
        }
    }

    //Recursively searches for every possible solution.
    void solveRecursive()
    {
        int emptyRow = -1;
        int emptyCol = -1;
        bool foundEmpty = false;
```

```
        //This d=find the first empty cell by scanning
        for (int row = 0; row < 9 && !foundEmpty; row++)
        {
            for (int col = 0; col < 9; col++)
            {
                if (board[row][col] == 0)
                {
                    emptyRow = row;
                    emptyCol = col;
                    foundEmpty = true;
                    break;
                }
            }
        }

        // If no empty cell remains, a complete solution was found.
        if (!foundEmpty)
        {
            solutionCount++;

            cout << "\nSolution " << solutionCount << ":\n";
            printBoard();
            return;
        }

        // Try every possible digit in the empty cell.
        for (int number = 1; number <= 9; number++)
        {
            if (isValid(emptyRow, emptyCol, number))
            {
                // Make a choice.
                board[emptyRow][emptyCol] = number;

                // Recursively solve the remaining empty cells.
                solveRecursive();

                // Backtrack: undo the choice so another number
                // can be tried and all solutions can be found.
                board[emptyRow][emptyCol] = 0;
            }
        }
    }
public:
    //This part initializes the board as empty and resets the solution count.
    SudokuSolver()
    {
```

```
        solutionCount = 0;

        for (int row = 0; row < 9; row++)
        {
            for (int col = 0; col < 9; col++)
            {
                board[row][col] = 0;
            }
        }
    }
    //Reads and validates a puzzle file.
    bool loadPuzzle(const string& filename)
    {
        ifstream inputFile(filename);

        if (!inputFile.is_open())
        {
            cout << "Error: Could not open " << filename << ".\n";
            return false;
        }

        string line;
        int row = 0;

        //Read the file one line at a time.
        while (getline(inputFile, line))
        {
            bool hasCharacter = false;

            for (char ch : line)
            {
                if (!isspace(static_cast<unsigned char>(ch)))
                {
                    hasCharacter = true;
                    break;
                }
            }

            if (!hasCharacter)
            {
                continue;
            }
            // A puzzle must have exactly nine rows.
            if (row >= 9)
            {
                cout << "Error: " << filename
```

```
                 << " has too many rows.\n";
            return false;
        }
        int col = 0;
        //Process every non-whitespace character in the row.
        for (char ch : line)
        {
            if (isspace(static_cast<unsigned char>(ch)))
            {
                continue;
            }

            //A row must contain exactly nine puzzle cells.
            if (col >= 9)
            {
                cout << "Error: A row in " << filename
                     << " has too many cells.\n";
                return false;
            }

            if (ch == '_')
            {
                board[row][col] = 0;
            }
            else if (ch >= '1' && ch <= '9')
            {
                board[row][col] = ch - '0';
            }
            else
            {
                cout << "Error: Invalid character in "
                     << filename << ".\n";
                return false;
            }

            col++;
        }

        if (col != 9)
        {
            cout << "Error: A row in " << filename
                 << " does not have nine cells.\n";
            return false;
        }

        row++;
```

```
        }

        inputFile.close();

        // The file must contain exactly nine nonblank rows.
        if (row != 9)
        {
            cout << "Error: " << filename
                 << " does not contain nine rows.\n";
            return false;
        }

        // Reject puzzles whose original clues already conflict.
        if (!hasValidStartingClues())
        {
            cout << "Error: " << filename
                 << " contains conflicting Sudoku clues.\n";
            return false;
        }

        return true;
    }

    //This part displays the puzzle and finds all its solutions.
    void solve(const string& filename)
    {
        cout << "\n==================================\n";
        cout << "File: " << filename << endl;
        cout << "==================================\n";
        cout << "Original puzzle:\n";
        printBoard();

        solutionCount = 0;
        solveRecursive();

        if (solutionCount == 0)
        {
            cout << "\nNo solution found.\n";
        }
        else
        {
            cout << "\nTotal solutions found: "
                 << solutionCount << endl;
        }
    }
};
```

```
int main()
{
    string filenames[5] =
    {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    //each puzzle gets a fresh SudokuSolver object.
    for (int i = 0; i < 5; i++)
    {
        SudokuSolver solver;

        if (solver.loadPuzzle(filenames[i]))
        {
            solver.solve(filenames[i]);
        }
    }

    return 0;
}
```